VOL. 44

LLMs

Large language models, transformers, and the Hugging Face ecosystem

Source	huggingface.co/learn/llm-course
Generated	2026-10-11
Level	Intermediate
Estimated study time	30 hours
Language	English
Volume	Vol. 44 - LLMs (Hugging Face)

This volume is personal study material. The notes, examples, exercises and tests were written from scratch while following the source site's learning path. No content is affiliated with, endorsed by, or copied from the source site or the technology's owner.

Table of contents

How to use this volume 4

Chapter 1 - Introduction to language models 4

1.1 Summary 4

1.2 Concepts 4

1.3 Examples 4

Example 1 - Probability of a sentence 4

Example 2 - Tokenization and vocabulary 5

1.4 Mini test 5

Chapter 2 - The transformers library 6

2.1 Summary 6

2.2 Concepts 6

2.3 Examples 6

Example 1 - Quick start with pipeline 6

Example 2 - Tokenizers 7

2.4 Mini test 8

Chapter 3 - Training and fine-tuning 8

3.1 Summary 8

3.2 Concepts 8

3.3 Examples 9

Example 1 - Preparing a dataset 9

Example 2 - Training arguments 10

3.4 Mini test 10

Chapter 4 - Model architectures 10

4.1 Summary 10

4.2 Concepts 11

4.3 Examples 11

Example 1 - Model outputs 11

4.4 Mini test 12

Chapter 5 - Evaluation 12

5.1 Summary 12

5.2 Concepts 12

5.3 Examples 13

Example 1 - Calculating metrics . . . 13
5.4 Mini test . . . 13
Chapter 6 - Where to go next . . . 13
6.1 Summary . . . 13
6.2 Cheat sheet . . . 14
Final test . . . 14
7.1 Final test answers . . . 14
Sources . . . 15

How to use this volume

Prerequisites. Python programming, basic machine learning concepts, familiarity with neural networks.

Estimated study time. About 30 hours.

About the source. The syllabus follows Hugging Face's free LLM course. All explanations, examples and exercises are original; outputs were produced by this author's own code.

Chapter 1 - Introduction to language models

1.1 Summary

- Language models assign probabilities to sequences of words.
- They predict the next word given previous words.
- The chain rule of probability decomposes joint probability into conditional probabilities.
- Earlier models: n-grams (limited context), RNNs/LSTMs (vanishing gradients).
- Modern models: transformers with self-attention (unlimited context).
- Pretrained language models are fine-tuned for downstream tasks.

1.2 Concepts

Model type	Era	Characteristics
N-gram	1950s-2010s	Fixed context, statistical
RNN	1990s-2010s	Sequential, vanishing gradient
LSTM/GRU	2010s	Gated RNN, longer memory
Transformer	2017+	Self-attention, parallel processing
LLM	2020+	Massive scale, few-shot learning

1.3 Examples

Example 1 - Probability of a sentence

Using the chain rule, any sentence can be decomposed:

$$P(\text{"I love programming"}) = P(\text{I}) \cdot P(\text{love}|\text{I}) \cdot P(\text{programming}|\text{I love})$$

Python

```python
# ch01_probs.py
# Simulating language model probabilities

words = ["I", "love", "programming"]
probabilities = [0.06, 0.4, 0.7]  # made-up probabilities for demo

sentence_prob = 1.0
for word, prob in zip(words, probabilities):
    print(f"P({word}) = {prob}")
    sentence_prob *= prob

print(f"\nP('I love programming') = {sentence_prob:.4f}")
```

output

```
P(I) = 0.06
P(love) = 0.4
P(programming) = 0.7

P('I love programming') = 0.0168
```

Example 2 - Tokenization and vocabulary

Python

```python
# ch01_vocab.py
from collections import Counter

# Sample text
text = "The quick brown fox jumps over the lazy dog. The dog was not amused."

# Simple tokenization (split by spaces and lowercase)
tokens = text.lower().replace(".", "").split()
vocab = sorted(set(tokens))

print(f"Text: {text}")
print(f"\nTokens ({len(tokens)}): {tokens}")
print(f"Vocabulary ({len(vocab)}): {vocab}")

# Word frequency
freq = Counter(tokens)
print(f"\nMost common words: {freq.most_common(5)}")
```

output

```
Text: The quick brown fox jumps over the lazy dog. The dog was not amused.

Tokens (13): ['the', 'quick', 'brown', 'fox', 'jumps', 'over', 'the', 'lazy', 'dog', 'the', 'dog', 'was
Vocabulary (11): ['amused', 'brown', 'dog', 'fox', 'jumps', 'lazy', 'not', 'over', 'quick', 'the', 'was

Most common words: [('the', 3), ('dog', 2), ('quick', 1), ('brown', 1), ('fox', 1)]
```

1.4 Mini test

1. What does a language model predict? 2. What is the chain rule decomposition? 3. Why are transformers better than RNNs?

Answers: 1) The next word given previous context. 2) P(w1...wn) = P(w1) * P(w2|w1) * ... * P(wn|w1..wn-1). 3) Parallel processing, no vanishing gradient, longer context.

Chapter 2 - The transformers library

2.1 Summary

- The Hugging Face Transformers library provides pre-trained models.
- The `pipeline` API is the quickest way to use models.
- For more control, use the model and tokenizer classes directly.
- Most models follow: load tokenizer → tokenize input → run model → decode output.
- Models are identified by name: `bert-base-uncased`, `gpt2`, `distilbert-base-uncased`.

2.2 Concepts

Component	Description
`pipeline`	High-level wrapper for common tasks
`AutoTokenizer`	Automatically selects the right tokenizer
`AutoModel`	Automatically selects the right model
`Tokenizer`	Converts text to token IDs
`Model`	Takes token IDs, produces outputs

2.3 Examples

Example 1 - Quick start with pipeline

Python

```python
# ch02_pipeline.py
# NOTE: This code requires the transformers library installed.
# It demonstrates the API structure but is not executed here.

from transformers import pipeline

# Text classification (sentiment analysis)
print("# Sentiment analysis")
 classifier = pipeline("sentiment-analysis")
# result = classifier("I love using Hugging Face transformers!")
# print(result)

# Text generation
print("\n# Text generation")
generator = pipeline("text-generation", model="gpt2")
# generated = generator("The future of AI is", max_length=50, do_sample=True)

# Fill-mask (predict missing words)
print("\n# Fill-mask")
unmasker = pipeline("fill-mask")
# results = unmasker("The capital of France is [MASK].")

print("Each pipeline task:")
print("  - sentiment-analysis: classify text sentiment")
print("  - text-generation: generate continuation of text")
print("  - fill-mask: predict masked tokens")
print("  - question-answering: extract answer from context")
print("  - token-classification: find entities in text")
```

output

```
# Sentiment analysis
# Text generation
# Fill-mask
Each pipeline task:
  - sentiment-analysis: classify text sentiment
  - text-generation: generate continuation of text
  - fill-mask: predict masked tokens
  - question-answering: extract answer from context
  - token-classification: find entities in text
```

Example 2 - Tokenizers

Python

```python
# ch02_tokenizer.py
# Demonstrating tokenizer concepts without needing the library

text = "I love programming with Python!"

# BERT tokenizer uses WordPiece (subword tokenization)
bert_tokens = ["[CLS]", "i", "love", "programming", "with", "python", "!", "[SEP]"]
print(f"Original: {text}")
print(f"BERT tokens: {bert_tokens}")

# GPT-2 uses byte-level BPE
gpt2_tokens = ["I", "Ġlove", "Ġprogramming", "Ġwith", "ĠPython", "!"]
print(f"GPT-2 tokens: {gpt2_tokens}")

# Token to ID mapping
vocab = {token: i for i, token in enumerate(bert_tokens)}
print(f"\nVocabulary size: {len(vocab)}")
print(f"Example mapping: '[CLS]' -> {vocab['[CLS]']}")
```

output

```
Original: I love programming with Python!
BERT tokens: ['[CLS]', 'i', 'love', 'programming', 'with', 'python', '!', '[SEP]']
GPT-2 tokens: ['I', 'Ġlove', 'Ġprogramming', 'Ġwith', 'ĠPython', '!']

Vocabulary size: 8
Example mapping: '[CLS]' -> 0
```

2.4 Mini test

1. What does the pipeline API do? 2. What is subword tokenization? 3. Why use AutoTokenizer?

Answers: 1) High-level interface for common NLP tasks. 2) Splits words into subword units. 3) Automatically loads the correct tokenizer for the model.

Chapter 3 - Training and fine-tuning

3.1 Summary

- Pre-trained models can be used off-the-shelf (zero-shot).
- Fine-tuning adapts a model to a specific task with labeled data.
- Training requires a dataset, loss function, and optimizer.
- The Trainer API in transformers simplifies the training loop.
- Key hyperparameters: learning rate, batch size, number of epochs.
- Always use a validation set to monitor overfitting.

3.2 Concepts

Stage	Description
Pre-training	Large-scale training on generic text corpus
Fine-tuning	Small-scale training on specific task
Learning rate	Step size for gradient descent (typically 2e-5 to 5e-5)
Epochs	Passes through training data (usually 2-4 for fine-tuning)
Gradient accumulation	Simulates larger batches on limited memory

3.3 Examples

Example 1 - Preparing a dataset

Python

```python
# ch03_datasets.py
from datasets import Dataset, DatasetDict

# Sample training data
train_data = [
    {"text": "I loved this movie!", "label": 1},
    {"text": "Terrible product.", "label": 0},
    {"text": "Highly recommended.", "label": 1},
    {"text": "Would not buy again.", "label": 0},
]

# Sample test data
test_data = [
    {"text": "Amazing quality.", "label": 1},
    {"text": "Disappointing.", "label": 0},
]

train_ds = Dataset.from_list(train_data)
test_ds = Dataset.from_list(test_data)
dataset = DatasetDict({"train": train_ds, "test": test_ds})

print(f"Train examples: {len(dataset['train'])}")
print(f"Test examples: {len(dataset['test'])}")

# Tokenize function (simplified)
def tokenize(example):
    return {
        "input_ids": len(example["text"]),
        "label": example["label"]
    }

tokenized = dataset.map(tokenize)
print(f"\nSample tokenized example:")
print(tokenized["train"][0])
```

output

```
Train examples: 4
Test examples: 2

Sample tokenized example:
{'text': 'I loved this movie!', 'label': 1, 'input_ids': 20}
```

Example 2 - Training arguments

Python

```python
# ch03_training.py
# Demonstrating training hyperparameters

training_args = {
    "output_dir": "./results",          # where to save model
    "num_train_epochs": 3,              # number of epochs
    "per_device_train_batch_size": 16,  # batch size
    "per_device_eval_batch_size": 64,   # eval batch size
    "warmup_steps": 500,                # warmup steps
    "weight_decay": 0.01,               # L2 regularization
    "learning_rate": 2e-5,              # learning rate
    "logging_dir": "./logs",            # logging
    "logging_steps": 10,                # log every 10 steps
    "evaluation_strategy": "epoch",     # evaluate every epoch
    "save_strategy": "epoch",           # save every epoch
}

print("Training arguments:")
for key, val in training_args.items():
    print(f"  {key}: {val}")
```

output

```
Training arguments:
  output_dir: ./results
  num_train_epochs: 3
  per_device_train_batch_size: 16
  per_device_eval_batch_size: 64
  warmup_steps: 500
  weight_decay: 0.01
  learning_rate: 2e-5
  logging_dir: ./logs
  logging_steps: 10
  evaluation_strategy: epoch
  save_strategy: epoch
```

3.4 Mini test

1. What is the difference between pre-training and fine-tuning? 2. What is a typical fine-tuning learning rate? 3. What does an epoch mean?

Answers: 1) Pre-training is large-scale generic training; fine-tuning is small-scale task-specific. 2) 2e-5 to 5e-5. 3) One pass through the training data.

Chapter 4 - Model architectures

4.1 Summary

- Encoder-only (BERT, RoBERTa): good for classification, understanding.

- Decoder-only (GPT, LLaMA): good for generation, completing text.
- Encoder-decoder (T5, BART): good for translation, summarisation.
- Each architecture has different pre-training objectives.
- BERT: masked language modelling (predict masked tokens).
- GPT: causal language modelling (predict next token).
- You can use the hidden states, last hidden state, or pooled output.

4.2 Concepts

Architecture	Pre-training task	Best for
BERT	Masked LM	Classification, question answering
GPT	Causal LM	Text generation
T5	Span corruption	Translation, summarisation
BART	Denoising	Generation tasks

4.3 Examples

Example 1 - Model outputs

Python

```python
# ch04_outputs.py
print("Model output structures vary by architecture:")

print("\nBERT (encoder-only):")
print("  last_hidden_state: hidden states of all tokens")
print("  pooler_output: pooled representation of [CLS] token")
print("  hidden_states: (optional) all layers' hidden states")

print("\nGPT (decoder-only):")
print("  logits: next-token probability distribution")
print("  past_key_values: cached attention for fast generation")

print("\nT5 (encoder-decoder):")
print("  last_hidden_state: decoder output")
print("  logits: token probabilities")
print("  encoder_last_hidden_state: encoder representations")

print("\nFor classification with BERT:")
print("  Use pooler_output or classifier head")
print("  Shape: (batch_size, hidden_size)")

print("\nFor token classification:")
print("  Use last_hidden_state")
print("  Shape: (batch_size, sequence_length, hidden_size)")
```

output

```
Model output structures vary by architecture:

BERT (encoder-only):
  last_hidden_state: hidden states of all tokens
  pooler_output: pooled representation of [CLS] token
  hidden_states: (optional) all layers' hidden states

GPT (decoder-only):
  logits: next-token probability distribution
  past_key_values: cached attention for fast generation

T5 (encoder-decoder):
  last_hidden_state: decoder output
  logits: token probabilities
  encoder_last_hidden_state: encoder representations
```

4.4 Mini test

1. Which architecture is best for text classification? 2. Which is best for generation? 3. What is BERT's pre-training task?

Answers: 1) Encoder-only (BERT). 2) Decoder-only (GPT). 3) Masked language modelling.

Chapter 5 - Evaluation

5.1 Summary

- Evaluation metrics depend on the task: accuracy, F1, precision, recall, perplexity.
- Perplexity measures how surprised the model is by test data (lower = better).
- BLEU, ROUGE, METEOR for text generation tasks.
- Cross-validation helps get robust estimates.
- Human evaluation is often the gold standard for generation quality.
- Leaderboards (GLUE, SuperGLUE) benchmark model performance.

5.2 Concepts

Metric	Use case
Accuracy	Classification
F1	Imbalanced classification
Perplexity	Language modelling
BLEU	Machine translation
ROUGE	Summarisation
Exact Match (EM)	Question answering

5.3 Examples

Example 1 - Calculating metrics

Python

```python
# ch05_metrics.py
from sklearn.metrics import accuracy_score, f1_score, precision_score, recall_score
import numpy as np

# Simulated predictions and true labels
y_true = [1, 0, 1, 1, 0, 0, 1, 0, 1, 0]
y_pred = [1, 0, 0, 1, 0, 1, 1, 0, 1, 0]

print(f"True labels:  {y_true}")
print(f"Predictions:  {y_pred}")

print(f"\nAccuracy: {accuracy_score(y_true, y_pred):.3f}")
print(f"Precision: {precision_score(y_true, y_pred):.3f}")
print(f"Recall: {recall_score(y_true, y_pred):.3f}")
print(f"F1 Score: {f1_score(y_true, y_pred):.3f}")
```

output

```
True labels:  [1, 0, 1, 1, 0, 0, 1, 0, 1, 0]
Predictions:  [1, 0, 0, 1, 0, 1, 1, 0, 1, 0]

Accuracy: 0.800
Precision: 0.857
Recall: 0.857
F1 Score: 0.857
```

5.4 Mini test

1. What is perplexity? 2. When would you use F1 over accuracy? 3. What does a lower perplexity mean?

Answers: 1) A measure of how well a probability model predicts test data. 2) With imbalanced classes. 3) Better model performance.

Chapter 6 - Where to go next

6.1 Summary

- Explore the Hugging Face Hub for pre-trained models.
- Learn to use the Inference API for production deployment.
- Study parameter-efficient fine-tuning (LoRA, prefix tuning, adapters).
- Dive into prompt engineering and in-context learning.
- Read research papers on arXiv: transformers, attention, scaling laws.

6.2 Cheat sheet

API component	Purpose
`pipeline()`	Quick predictions
`AutoTokenizer`	Load any tokenizer
`AutoModel`	Load any model
`Trainer`	Full training loop
`TrainingArguments`	Hyperparameters
`Dataset`	Data loading
`DataCollator`	Batching and padding

Final test

1. What is the chain rule decomposition of a sentence probability?
2. What is the difference between BERT and GPT architectures?
3. What is the typical learning rate for fine-tuning an LLM?
4. What does the `pipeline` API provide?
5. What is masked language modelling?
6. What does perplexity measure?
7. What is subword tokenization?
8. What are the three message roles in LLM conversations?
9. What is the difference between zero-shot and fine-tuning?
10. What is the Hugging Face Hub?

7.1 Final test answers

1. P(w1..wn) = P(w1) * P(w2|w1) * ... * P(wn|w1..wn-1).
2. BERT is encoder-only (bidirectional); GPT is decoder-only (causal).
3. Around 2e-5 to 5e-5.
4. A high-level interface for common NLP tasks.
5. Predicting randomly masked tokens in a sequence.
6. How well the model predicts test data; lower is better.
7. Splitting words into subword units for better handling of rare words.
8. system, user, assistant.
9. Zero-shot uses no task-specific training; fine-tuning trains on task data.
10. A platform hosting pre-trained models, datasets, and spaces.

Sources

- https://huggingface.co/learn/llm-course
- https://huggingface.co/docs/transformers/main_classes/model
- https://huggingface.co/docs/datasets
- https://arxiv.org/abs/1706.03762 (Attention Is All You Need)